

第9回: オリジナルアプリ制作

Webアプリケーション基礎 2026

今日のゴール

完成したTODOアプリを土台に、**自分だけのオリジナルアプリ**を作る

8回の授業でTODOアプリが完成しました。

- HTML / CSS / JavaScript (ブラウザ側)
- Python / FastAPI / SQLite (サーバー側)
- Fetch APIによる結合、セキュリティ対策

今日はこの完成品を「改造」して、自分の題材のアプリに作り変えます。
ゼロから書くのではなく、**動くものを少しずつ変えていく**のがポイントです。

今日の流れ

前半

- 考え方: 構造は変えない、題材を変える
- 設計: データから考える (テーブル → API → 画面)
- バックエンドの改造
- フロントエンドの改造
- レベルアップ (カラム追加・機能追加)
- セキュリティ点検・最終仕上げ・完成デモ

1. 考え方

構造は変えない、題材を変える

TODOアプリの正体

TODOアプリは、実は**Webアプリの最小完全形**です。

層	ファイル	役割
画面	<code>static/index.html</code> / <code>style.css</code>	構造と見た目
動き	<code>static/app.js</code>	入力を受け取り、APIを呼び、画面を描く
API	<code>main.py</code> (FastAPI)	データの追加・取得・更新・削除
保存	<code>todo.db</code> (SQLite)	データを永続化

「**1種類のデータを、作って・見て・更新して・消す**」(CRUD)

世の中のWebアプリの多くは、この形の組み合わせでできています。

構造は変えない、題材を変える

your-app 制作の基本方針:

- 変えないもの: 3層構成 (ブラウザ $\rightleftharpoons$ FastAPI $\rightleftharpoons$ SQLite)、CRUDの4つのAPI、ファイル構成
- 変えるもの: 題材 (何を記録するか)、テーブルのカラム、画面の見た目

なぜこの方針か:

1. ゼロから書くと、どこが壊れたか分からなくなる
2. 完成品を小さく変えて → 動作確認 → **commit** すれば、常に動く状態を保てる
3. 「TODO」が自分の題材に置き換わる過程で、**全部の層のつながり**が復習できる

題材の例

「何を記録・管理するアプリ？」を一文で言えるものを選びます。

題材	記録するもの	状態（チェックボックス）
読書記録	本のタイトル	読了した / まだ
部活の練習記録	練習メニュー	達成した / まだ
欲しいもののリスト	商品名	買った / まだ
映画メモ	映画のタイトル	観た / まだ
宿題管理	宿題の内容	提出した / まだ

ポイント: 「内容 + 済み/未済み」の形に当てはまる題材が作りやすい
(TODOの `title` と `done` にそのまま対応するため)

2. 設計

データから考える

設計はデータから考える

授業ではHTML（第2回）から学びましたが、設計するときはデータから考えます。

手順	問い	対応する授業
1. 題材	何を記録するアプリ？	第1回
2. テーブル	どんなカラムが必要？	第6回（SQLite）
3. API	URLとメソッドは？	第5・6回（FastAPI, REST）
4. HTML	入力欄はどれ？	第2回
5. CSS	どんな見た目？	第3回
6. JS	何をfetchして何を描く？	第4・7回
7. 点検	安全か？	第8回

データの形が決まれば、API・画面は自然に決まります。

テーブル設計

TODOアプリのテーブルを思い出しましょう:

```
CREATE TABLE IF NOT EXISTS todos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT, -- 自動で増える番号  
  title TEXT NOT NULL, -- 内容  
  done INTEGER DEFAULT 0 -- 状態 (0/1)  
)
```

これを自分の題材に置き換えます。例: 読書記録アプリ

```
CREATE TABLE IF NOT EXISTS books (  
  id INTEGER PRIMARY KEY AUTOINCREMENT, -- そのまま  
  title TEXT NOT NULL, -- 書名  
  finished INTEGER DEFAULT 0 -- 読了したか (0/1)  
)
```

まずは「id + 内容カラム1個 + 状態カラム1個」の3つで始める

API設計

テーブルが決まれば、APIは機械的に決まります。例: 読書記録アプリ

TODOアプリ	読書記録アプリ	役割
GET /todos	GET /books	一覧を取得
POST /todos	POST /books	新規追加
PUT /todos/{id}	PUT /books/{id}	状態を更新
DELETE /todos/{id}	DELETE /books/{id}	削除

REST APIの設計ルール（第6回）はそのまま:

- URLは**名詞の複数形**（ /books , /movies , /homeworks ）
- 操作の種類は**HTTPメソッド**で表す（GET / POST / PUT / DELETE）

変換表を作る

改造を始める前に、対応表を作っておくと迷いません。

項目	TODOアプリ	自分のアプリ（例: 読書記録）
アプリ名	TODO App	読書記録 App
テーブル名	<code>todos</code>	<code>books</code>
内容カラム	<code>title</code>	<code>title</code> （書名）
状態カラム	<code>done</code> （完了）	<code>finished</code> （読了）
DBファイル	<code>todo.db</code>	<code>books.db</code>
APIのURL	<code>/todos</code>	<code>/books</code>

この表の左の単語を、全ファイルから探して右に置き換えていくのが改造作業です。

実習1: 題材を決めて設計する

やること

1. `your-app/` フォルダに `design.md` を新規作成
2. 以下の3つを書く:

私のアプリ設計

1. 題材（一文で）

例: 読んだ本と感想を記録するアプリ

2. テーブル設計

テーブル名: `books`

カラム: `id` / `title` (書名) / `finished` (読了 0 or 1)

3. 変換表

`todos` → `books`, `done` → `finished`, `todo.db` → `books.db`, `/todos` → `/books`

ポイント: まだコードは触らない。設計が決まってから手を動かす

3. バックエンドの改造

main.py を自分の題材に置き換える

「title」を追いかける

改造の本質はリネームです。 `todos` や `done` という単語が、どのファイルのどこに現れるかを追いかけます。

場所	何をしているか	学んだ回
<code>main.py</code> の <code>CREATE TABLE</code>	テーブルとカラムの定義	第6回
<code>main.py</code> の SQL (<code>SELECT/INSERT/...</code>)	データの読み書き	第6回
<code>main.py</code> の Pydanticモデル	受け取るデータの形	第5・8回
<code>main.py</code> の <code>@app.get("/todos")</code> など	APIのURL	第5回
<code>app.js</code> の <code>API_URL</code> と <code>fetch</code>	APIの呼び出し	第7回
<code>app.js</code> の描画処理	データを画面に反映	第4回
<code>index.html</code> のフォーム・見出し	入力欄とラベル	第2回

1つのデータが全部の層を通る——第7回で学んだ流れを、自分の手で作り直します。

main.py の変更ポイント

```
DATABASE = "books.db"          # ① DBファイル名

def init_db():
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS books (    -- ② テーブル名
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            finished INTEGER DEFAULT 0          -- ③ カラム名
        )
    """)

class BookUpdate(BaseModel):    # ④ Pydanticモデル
    finished: bool

@app.get("/books")              # ⑤ URLと関数名
def get_books():
    ...
```

SQLの中の `todos` / `done` もすべて置き換えます。1か所でも残ると動きません。

注意: テーブルを変えたら古いDBを消す

`CREATE TABLE IF NOT EXISTS` は「無ければ作る」でした（第6回）。

つまり、古い `todo.db` が残っていると、新しいテーブルは作られません。

テーブル名やカラムを変えたら:

```
# サーバーを止めて (Ctrl+C) から
rm todo.db           # 古いDBファイルを削除
python main.py       # 再起動すると新しいテーブルが作られる
```

DBファイルを消すとデータも消えます。学習用アプリなので今は気にせずOK。
本物のサービスでは「マイグレーション」という仕組みで安全に変更します。

4. フロントエンドの改造

画面を自分の題材に置き換える

HTML: 入力欄はカラムと対応する

フォームの入力欄（第2回）は、テーブルのカラムと1対1で対応します。

```
<title>読書記録 App</title>           <!-- タブの表示名 -->
<h1>読書記録 App</h1>                 <!-- 見出し -->
<input
  type="text"
  id="todo-input"
  placeholder="読んだ本のタイトルを入力..." <!-- 案内文 -->
  maxlength="100"
/>
<button type="submit">追加</button>
```

ポイント: `id="todo-input"` などのidは、`app.js` が
`document.getElementById` で探しに来る名前（第4回）。
変える場合は **HTMLとJSの両方**を揃えて変えること。

JavaScript: fetch先と描画を合わせる

app.js の変更ポイントは大きく2つ:

```
// ① APIのURL (第7回: fetchの呼び出し先)
const API_URL = "/books";

// ② サーバーから返るデータのカラム名 (第4回: DOM操作)
titleSpan.textContent = book.title;
checkbox.checked = book.finished;          // done → finished
checkbox.addEventListener("change",
  () => toggleBook(book.id, book.finished));
```

サーバーが `{"id": 1, "title": "...", "finished": false}` を返すなら、JS側も `finished` で受け取る——**APIとフロントの「約束」** (第7回) です。

開発者ツールの**Network**タブで、実際に飛んでいるリクエストとJSONを確認しながら進めましょう。

CSS: 個性はここで出す

CSS（第3回）は壊れにくく、一番差がつく場所です。

変えるもの	例
配色	背景色・ボタン色をテーマカラーに
フォント	<code>font-family</code> を変える
角丸・影	<code>border-radius</code> / <code>box-shadow</code>
レイアウト	カードの幅、余白の調整

```
.header {  
  background-color: #1e3a2f; /* 読書アプリなら深緑に */  
}  
.todo-button {  
  background-color: #2d6a4f;  
}
```

ポイント: 機能がすべて動いてから凝るのがおすすめ。見た目は後からいくらでも変えられる

5. レベルアップ

カラム追加・機能追加に挑戦する

3つのレベル

レベル	内容	目安
Level 1	題材・名前・見た目の変更（ここまでの実習）	全員必達
Level 2	カラムを1つ追加（感想、期限、点数など）	挑戦しよう
Level 3	機能を1つ追加（件数表示、絞り込みなど）	余裕があれば

さらに発展（授業の範囲外。Level 3を達成したらこちらもおすすめ）：

- ログイン・ユーザー登録（認証）
- テーブルを2つ以上使う（JOIN）
- 画像のアップロード
- 外部のAPIを呼ぶ

まず「小さくても全部自分で説明できるアプリ」を完成させることが目標です。

Level 2: カラム追加は「全層ツアー」

例: 読書記録に `comment` (感想) カラムを追加する場合の変更マップ

層	ファイル	変更内容
DB	<code>main.py</code>	<code>CREATE TABLE</code> に <code>comment TEXT</code> を追加 (+ 古いDB削除)
入力の形	<code>main.py</code>	<code>BookCreate</code> に <code>comment: str = Field(max_length=200)</code>
API	<code>main.py</code>	<code>INSERT / SELECT</code> に <code>comment</code> を追加、返す辞書にも追加
画面	<code>index.html</code>	感想の <code><input></code> を1つ追加
動き	<code>app.js</code>	<code>POST</code> のbodyに <code>comment</code> を含め、一覧に表示する

カラム1つで、第2回～第8回の内容をすべて触ることになります。
これがLevel 2の学習効果です。

Level 3: 機能追加の例

新しいエンドポイントや表示を1つ追加します。

機能	作り方のヒント
件数の表示	<code>todos.length</code> を画面に表示（JSだけで完結）
済み/未済みで絞り込み	一覧を <code>filter()</code> してから描画（第4回）
並び順の変更	SQLの <code>ORDER BY</code> を変える（第6回）
全部済みにするボタン	新しいエンドポイント <code>PUT /books/finish-all</code> を作る

進め方のコツ:

1. 「どの層の変更で実現できるか?」をまず考える（JSだけ? SQLだけ? 全部?）
2. 変更する層が少ない機能から着手する
3. 1機能できるごとに動作確認 → commit

6. 仕上げ

セキュリティ点検と完成デモ

セキュリティ4点チェック

第8回で学んだ対策が、改造後も生きているか点検します。

チェック	確認する場所	学んだ回
① XSS対策	<code>app.js</code> で <code>textContent</code> を使っている (<code>innerHTML</code> 禁止)	第8回
② SQLインジェクション対策	SQLはすべて <code>?</code> のパラメータバインディング	第8回
③ バリデーション	Pydanticの <code>Field(min_length=1, max_length=100)</code>	第8回
④ エラーハンドリング	404を返す / try-catchでエラーメッセージ表示	第8回

特に注意: カラムを追加した人は、

新しいカラムにも ①～③ が効いているか確認すること

(新しい `Field` 制約を付けたか? 表示に `textContent` を使ったか?)

セキュリティ点検

やることの例

自分のアプリに攻撃者のつもりで入力してみます。

1. **XSS:** `<script>alert('XSS')</script>` を入力して追加
 - 。そのまま文字として表示されればOK（実行されたらNG）
2. **バリデーション:** 空文字で追加 → エラーメッセージが出ればOK
3. **バリデーション:** 101文字以上を `/docs` から送信 → 422エラーならOK
4. **エラーハンドリング:** `/docs` から存在しないid（例: 9999）を更新・削除
 - 。404が返り、画面ではエラーメッセージが表示されればOK

第8回の実習と同じ手順です。自分で改造したコードでも守れているかがポイント。

保存方法

```
git add -A  
git commit -m "変更した内容を書く"  
git push
```